[95] Linyi Yang, Tin Lok James Ng, Barry Smyth, and Riuhai Dong. Html: Hierarchical transformer-based multi-task learning for volatility prediction. In *Proceedings of The Web Conference 2020*, pages 441–451, 2020.

[96] Yupeng Cao, Zhi Chen, Qingyun Pei, Prashant Kumar, KP Subbalakshmi, and Papa Momar Ndiaye. Ecc analyzer: Extract trading signal from earnings conference calls using large language model for stock performance prediction. *arXiv preprint arXiv:2404.18470*, 2024.

[97] John C Hull. *Options, Futures, and Other Derivatives*. Pearson Education, 2017.

[98] Xiao-Yang Liu, Hongyang Yang, Jiechao Gao, and Christina Dan Wang. FinRL: Deep reinforcement learning framework to automate trading in quantitative finance. *ACM International Conference on AI in Finance (ICAIF)*, 2021.

[99] Xiao-Yang Liu, Ziyi Xia, Jingyang Rui, Jiechao Gao, Hongyang Yang, Ming Zhu, Christina Wang, Zhaoran Wang, and Jian Guo. Finrl-meta: Market environments and benchmarks for data-driven financial reinforcement learning. *Advances in Neural Information Processing Systems*, 35:1835–1849, 2022.

[100] Volodymyr Mnih, Adria Puigdomenech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In *International conference on machine learning*, pages 1928–1937. PMLR, 2016.

[101] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[102] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. Playing atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602*, 2013.

[103] Yuliya Plyakha, Raman Uppal, and Grigory Vilkov. Why does an equal-weighted portfolio outperform value-and price-weighted portfolios? *Available at SSRN 2724535*, 2012.

[104] Jaap MJ Murre and Joeri Dros. Replication and analysis of ebbinghaus' forgetting curve. *PloS one*, 10(7):e0120644, 2015.

[105] Guardrails ai. `https://docs.guardrailsai.com`. Open source library for interacting with Large Language Models.

A Appendix

A.1 Related Work

LLM Agents for Financial Decision Making. There are considerable efforts towards developing general-purpose LLM agent for sequential decision-making [49, 50], and such type of tasks often involve episodic interactions with environment and verbal reflections for action refinement, such as coding competition [51, 52], software development [53, 23], game-playing [54, 55]. Furthermore, researchers have started to exploit how LLM agents can perform better in harder decision-making tasks from finance [56, 57, 58, 59, 60], in which there are more volatile environments, leading to that the numerous unpredictable elements can obscure an agent's ability to reflect accurately on the reasons for poor decision outcomes. FinMem [32] enhances single stock trading performance by embedding memory modules with LLM agent for reflection-refinement, and FinAgent [33] improved trading profits via using external quantitative tool to fight against volatile environment.

Multi-Agent System and Communication Structures. In traditional multi-agent systems [61, 62], the way for agents' communication is pre-determined, like sharing data or state observations [63, 64, 65, 66, 67, 68]. The emergence of large language model brings flexibility for human-understandable communications [69, 20, 23, 70], so some work tries to elevate decision-making ability of LLM-based multi-agent system by letting agents engage in discussions [71, 21] or debates [72, 73]. The similar peer-communication strategy was as well utilized by the multi-agent system for financial tasks [74, 75, 76]. However, such approach are not optimal for unified-goal financial tasks that prioritize profits [77], because they suffer from potentially ambiguous optimization objectives and are unable to control the unnecessary communication costs [78].

Prompt Optimization and Verbal Reinforcement. To enhance the reasoning or decision-making of LLM agents, many prompt optimization techniques have been proposed, like ReAct [79], Chain of Thought (CoT) [80], Tree of Thoughts (ToT) [81], ART [14], intended for that LLM agents can automatically generate intermediate reasoning steps as an iterative program. In addition, to make LLM agents make decisions like humans and generate more understandable reasoning texts, some researchers recommend incorporating cognitive structures [82, 83, 44, 84]. Inspired by these previous work and DRL algorithms [85, 86, 87, 67, 88], verbal reinforcement [29, 30, 89, 24] was developed for LLM agents such that they can update actions based on iterative self-reflection while integrating additional LLM as a prompt optimizer [27, 28].

A.2 Textual Gradient-Descent

In an LLM-based prompt optimizer, a meta-prompt [27, 28] is used to refine the task prompt for better performance. For example, for a mathematical reasoning task, the task prompt might be "Let's solve the problem," while the meta-prompt could be "Improve the prompt to help a model better perform mathematical reasoning."

Although prompt optimization lacks explicit gradients to control the update direction, we can simulate "textual gradient" by using LLMs' reflection capabilities. By generating feedback from past successes and failures on trading decisions, LLMs can produce "semantic" gradient signals that guide the optimization process.

Adjusting the optimization process's direction is crucial, similar to tuning the learning rate in traditional parameter optimization. An inappropriate learning rate can cause the process to oscillate or converge too slowly. Similarly, without proper control, the LLM-based optimizer might overshoot or oscillate during prompt optimization.

To mimic learning rate effects, we measure the overlapping percentage between trading decision sequences from consecutive iterations. We then directly edit the previous task prompt to enhance performance. The meta-prompt instructs the LLM to modify the current prompt based on feedback, ensuring a stable and incremental improvement process. This method allows for effective exploitation of existing prompts, leading to gradual performance enhancement.

A.3 FINCON Testing Stage Workflow

During the testing stage, FINCON will utilize the investment beliefs learned from the training stage, and the over-episode risk control mechanism will no longer operate. However, the within-episode risk control mechanism will still function, allowing the manager agent to adjust trading actions in real-time based on short-term trading performance and market fluctuations. This ensures that even during testing, FINCON can promptly respond to market risks and potentially prevent losses while leveraging the knowledge gained during training.

Algorithm 2 Testing Stage Algorithm of FINCON

Initialize trading start date s, stock pool of portfolio and portfolio weights $w_0 = \mathbf{0}$.
Inherit manager-analysts component $\{M_{pr}^i\}_{i=1}^I \& M_a$.
Inherit the reflections B, learned prompts $\boldsymbol{\theta}$, the trained policy $\Pi_{\boldsymbol{\theta}}$.
for $T + 1 \leq t \leq S$ **do**
  Run policy $\Pi_{\boldsymbol{\theta}}$ (collecting daily PnL r_t, portfolio weights w_t and daily CVaR value ρ_t).
  if $\rho_t < \rho_{t-1}$ **or** $r_t < 0$ **then**
    Trigger M_a self-reflection.
  end if
  Get one investment trajectory $\mathcal{H}$.
end for
Output performance metrics calculation results based on $\mathcal{H}$.